



**FACULTAD
DE INGENIERIA**

Universidad de Buenos Aires

AN I (7512) - AN I (9504) - MN (CB051)

TRABAJO PRÁCTICO 2

1^{ER} CUATRIMESTRE 2026

Interpolación e integración numérica de órbitas

Curso 5 - Cátedra Sassano.
26 de mayo de 2026

Resumen: El trabajo práctico se centrará alrededor de un problema básico de determinación de la órbita de un satélite tipo LEO (Low-Earth orbit). Se plantearán dos problemas a resolver, a saber:

- disponiendo de un conjunto acotado de posiciones discretas de un satélite, se requiere implementar un programa donde se pueda generar cualquier posición intermedia de este.
- a partir de un modelo simplificado de la dinámica orbital de un satélite LEO, se requiere disponer de un programa que calcule las posiciones orbitales de dicho satélite a partir de una posición y velocidad dadas.

Instrucciones de entrega: Por campus un único archivo comprimido ZIP. Debe incluir las instrucciones para que los docentes puedan ejecutar su código.

Fecha límite de entrega: 25 de junio 2026 23:59 hs.

- El trabajo entregado en dicha fecha deberá tener completo los items que no son opcionales (indicado en el enunciado como “(opcional)”). No se admitirán entregas parciales en ningún ejercicio. No se admitirán prórrogas.
- Solo en los casos que deban realizar correcciones menores, se admitirá una sola entrega adicional, no implicando la aceptación de entregas parciales en primera instancia. La desaprobación del trabajo implica la desaprobación de la cursada.
- Si bien se alienta a los estudiantes a discutir resultados obtenidos, se recuerda que cualquier forma de plagio o copia implicará la desaprobación de la cursada de forma automática para todos los integrantes del grupo.

1. SW interpolador de órbitas LEO

- A Diseñar e implementar un programa en Python3 que permita interpolar un conjunto dado de posiciones orbitales provistas. Se asume como hipótesis que dichas posiciones se corresponden a la dinámica orbital de un satélite LEO lo suficientemente lentas como para poder utilizar una spline cúbica natural.

El usuario de dicho programa le proveerá un archivo de texto con la siguiente información:

```
FECHA HORA POS_X POS_Y POS_Z VEL_X VEL_Y VEL_Z
```

Las posiciones tienen unidades de [km] y las velocidades de [km/s].

- B Un colega se encuentra trabajando en un algoritmo de estimación de posición para el satélite SAC-D (Click link Wikipedia SAC-D) a partir de mediciones obtenidas por receptores GPS abordo del mismo. Lamentablemente el satélite no posee suficiente capacidad de cómputo y dicho algoritmo solamente puede proveer estimaciones cada 10 minutos. Dado que ustedes están por aprobar Análisis Numérico, sugieren utilizar su interpolador recientemente programado para proveer estimaciones de posición a una tasa mayor sin incurrir en una sobrecarga computacional para la computadora abordo del satélite. Entonces, su colega les provee el archivo “SACD_TPV_step_600s.txt” con estimaciones de posición cada 10 minutos del SAC-D, y les solicita graficar 1 periodo orbital (aproximadamente 110 minutos) superponiendo las muestras utilizadas para interpolar de forma que sea posible verificar visualmente si la interpolación es correcta. Por otro lado, les solicita que en un gráfico separado superpongan cada componente de la posición interpolada con las muestras utilizadas.

- C Al cabo de unas semanas¹ su colega desarrolla un algoritmo para proveer estimaciones precisas cada 1 segundo, aumentando el uso de la computadora de abordo del SAC-D, pero sin necesidad de cambiarla por una mejor. Este les entrega el archivo “SACD_TPV_step_1s.txt” con las posiciones precisas estimadas cada 1 segundo, y les solicita que comparen dichas posiciones con aquellas obtenidas de muestrear cada 1 segundo las splines del punto anterior (es decir, utilizando datos cada 10 minutos). Para ello, en un mismo gráfico, superponer las diferencias en cada componente de posición para tener una estimación aproximada del error.

¿Qué observa?

2. SW propagador de órbitas LEO

Asumiendo que la Tierra y un satélite LEO son objetos puntuales, con ciertas masas m_{Tierra} y $m_{satelite}$, y que la única fuerza presente es la atracción entre ellos, entonces las posiciones y velocidades del satélite quedan completamente determinadas a partir de la siguiente ecuación

¹y unas cuantas aspirinas...

diferencial

$$\ddot{\mathbf{r}} = -\frac{\mu}{r^3}\mathbf{r} \quad \forall \mathbf{r} \in \mathcal{R}^3 \quad (2.1)$$

$$\mu \approx G \cdot m_{Tierra} \quad (2.2)$$

con condiciones iniciales en posición y velocidad conocidas $\mathbf{r}$ y $\dot{\mathbf{r}}$ respectivamente, y con G la constante de gravitación universal.

- A Implementar en Python3 un integrador numérico Runge-Kutta 4 para resolver dicha ecuación diferencial.
- B Para cada una de las siguientes condiciones iniciales correspondientes a diferentes tipos de órbitas, mostrar una sola órbita de cada tipo en un gráfico de forma superpuesta. Utilice un paso de 1 segundo.

Órbita circular:

$$P_0 = [6125,24 \quad -3547,04 \quad -3,31277]^t \text{ km} \quad (2.3)$$

$$V_0 = [-0,519174 \quad -0,903482 \quad 7,43159]^t \text{ km/s} \quad (2.4)$$

Órbita geoestacionaria:

$$P_0 = [-41974,9 \quad 4012,93 \quad 23,5515]^t \text{ km} \quad (2.5)$$

$$V_0 = [-0,292606 \quad -3,06063 \quad 0,000293508]^t \text{ km/s} \quad (2.6)$$

Órbita Molniya:

$$P_0 = [305,926 \quad 3162,83 \quad 6324,79]^t \text{ km} \quad (2.7)$$

$$V_0 = [-9,86976 \quad 0,326984 \quad 0,313879]^t \text{ km/s} \quad (2.8)$$

- C (opcional) Para comprobar si el integrador funciona correctamente, propagar durante 24hs a un paso de 1 segundo, cada tipo de órbita definida por las condiciones iniciales del punto anterior y calcular el momento angular, su derivada, y la energía total del satélite, dadas respectivamente por las siguientes expresiones:

$$\mathbf{h} = \mathbf{r} \wedge \dot{\mathbf{r}} \quad (2.9)$$

$$\dot{\mathbf{h}} = \dot{\mathbf{r}} \wedge \dot{\mathbf{r}} + \mathbf{r} \wedge \ddot{\mathbf{r}} \quad (2.10)$$

$$\varepsilon = \frac{1}{2}\dot{\mathbf{r}} \cdot \dot{\mathbf{r}} - \frac{\mu}{r} \quad (2.11)$$

Realizar 3 gráficos mostrando cada una de dichas magnitudes, donde en cada gráfico se superponen las órbitas para cada tipo de condición inicial. Tenga en cuenta que el modelo dinámico que describe la ecuación diferencial implica que el momento angular y energía se conservan.

¿Si aumenta el paso de integración y el tiempo de propagación, qué sucede con dichas variables calculadas?

D (opcional) Ahora que dispone de un integrador, decide verificar si puede mejorar la solución al problema de estimación precisa de la órbita del SAC-D. Para ello, tome como condición inicial la primer estimación del archivo “SACD_TPV_step_600s.txt” y propague 1 órbita con su integrador utilizando un paso de 10 minutos. Utilizando el interpolador desarrollado, interpole la trayectoria generada por el integrador y luego realice un gráfico comparando dicha interpolación con aquella producida en el ejercicio 1.B. Se sugiere mostrar las diferencias de las normas de la posición. Por último, reduzca el paso de integración a 1 segundo y nuevamente realice un gráfico comparativo entre las normas de posición interpoladas a 10 minutos y las integradas e interpoladas a 1 segundo.

¿Qué observa?

3. SW propagador de órbitas LEO con correcciones GPS (opcional)

Hubo un imprevisto en el plan de proyecto del desarrollo del satélite SAC-D y ahora se le exige a su colega que libere recursos de la computadora de abordo del satélite para que sean utilizados por otras funcionalidades críticas de la plataforma. De esta forma, las estimaciones precisas de posición que produce su colega utilizando mediciones GPS ahora se realizan a una tasa de 1 estimación por minuto. A pesar de esta limitante, se requiere disponer de soluciones de posición precisas cada 1 segundo.

Utilizando el integrador numérico, ustedes sugieren corregir los errores producidos por el integrador utilizando las estimaciones de su colega cada 1 minuto, provistas en el archivo “SACD_TPV_step_60s.txt”. Para ello plantean el siguiente esquema:

- A partir de una estimación inicial con datos GPS, propagar la órbita del satélite con paso de 1 segundo hasta la próxima estimación de posición precisa (integrar por 60 segundos).
- Cada vez que se obtiene una estimación precisa con datos GPS, reiniciar el integrador y utilizar como condición inicial dicha estimación.
- El algoritmo termina cuando ya no hay más estimaciones por GPS disponibles.

Por lo tanto, se les solicita implementar en Python3 dicho esquema de estimación con correcciones GPS, y evaluar si se producen o no mejoras respecto del esquema de estimación de posición precisa a una tasa de 1 estimación por segundo que había planteado originalmente su colega. Se sugiere realizar un gráfico de la diferencia de las normas de las posiciones estimadas a 1 segundo utilizando el nuevo esquema propuesto y de las posiciones estimadas provistas por el algoritmo original de su colega en “SACD_TPV_step_1s.txt”.

¿Qué observa?

4. Funciones de biblioteca permitidas

Para facilitar la implementación, utilizar las siguientes funciones que proveen las bibliotecas de Python. No está permitido utilizar otras funciones de biblioteca, en particular ninguna que resuelva sistemas lineales o interpolaciones. Si tiene dudas, consulte a los docentes.

- Para calcular normas: <https://numpy.org/doc/stable/reference/generated/numpy.linalg.norm.html>
- Para verificar que no se esté dividiendo por un número casi nulo: <https://docs.python.org/dev/library/math.html#math.isclose>
- Para multiplicación de matrices: <https://numpy.org/doc/stable/reference/generated/numpy.dot.html>
- Función piso: <https://numpy.org/doc/stable/reference/generated/numpy.floor.html>
- Para cargar archivos de texto: <https://numpy.org/doc/stable/reference/generated/numpy.genfromtxt.html>
- Para producir muestras: <https://numpy.org/doc/stable/reference/generated/numpy.linspace.html>
- Para producir gráficos en 3D: https://matplotlib.org/stable/api/_as_gen/mpl_toolkits.mplot3d.axes3d.Axes3D.html.
Ejemplo:

```
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = plt.axes(projection='3d')
```
- Para producir scatter plots en 3D: <https://matplotlib.org/stable/gallery/mplot3d/scatter3d.html>
- Para producir scatter plots en 2D: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.scatter.html